



Production-Style Serverless Backend

using AWS Lambda & DynamoDB

Full CRUD Operations: Create, Read, Update, Delete

A scalable, secure and cost-effective serverless backend for managing user data

AWS Cloud & DevOps Learning Journey

Project Introduction



Learning Focus

This project demonstrates hands-on understanding of production-style serverless architecture concepts through practical implementation. It represents a learning journey into modern cloud backend development using AWS services.



Project Scope

A serverless backend system that handles Create, Read, Update, and Delete (CRUD) operations using AWS Lambda functions integrated with DynamoDB. The architecture follows production-ready design patterns while acknowledging this is a learning implementation focused on core concepts.



Technologies Used

AWS Lambda (Python), Amazon DynamoDB, Amazon API Gateway, AWS IAM, Amazon CloudWatch, Amazon SNS, and Boto3 SDK for AWS service interactions.

Problem Statement

Traditional Backend Challenges

- **Server Management**
Requires provisioning, configuring, and maintaining servers
- **Scaling Issues**
Manual scaling during traffic spikes
- **Cost Inefficiency**
Pay for idle server capacity 24/7
- **Operational Overhead**
Security patches, monitoring, backups

Serverless Solution

- **No Server Management**
AWS manages infrastructure completely
- **Auto-Scaling**
Automatically scales with demand
- **Pay-Per-Use**
Only pay for actual execution time
- **Built-in Availability**
High availability and fault tolerance

Why Serverless Architecture?



Zero Server Management

No need to provision or maintain servers. AWS handles all infrastructure operations.



Automatic Scaling

Scales automatically from zero to thousands of requests without manual intervention.



Cost Efficiency

Pay only for compute time used. No charges when code isn't running.



Built-in Security

IAM-based permissions and AWS managed security features protect your application.

Traditional Backend vs Serverless Backend

Infrastructure	Manual server provisioning and management	Fully managed by cloud provider
Scaling	Manual scaling configuration required	Automatic scaling based on demand
Cost Model	Pay for reserved capacity (24/7)	Pay only for execution time
Deployment	Complex deployment pipelines	Simple function deployment
Maintenance	OS patches, security updates, monitoring	Managed by cloud provider
Availability	Requires load balancers, multiple instances	Built-in high availability

AWS Services Used



AWS Lambda

Serverless compute service that runs Python code in response to events



Amazon DynamoDB

NoSQL database for fast and flexible data storage with auto-scaling



Amazon API Gateway

RESTful API endpoints to route HTTP requests to Lambda functions



AWS IAM

Identity and access management for secure permissions between services



Amazon CloudWatch

Monitoring and logging service for tracking Lambda execution metrics



Amazon SNS

Simple notification service for alerts and notifications

Architecture Overview

The architecture diagram represents the intended production-style serverless workflow, while the current implementation primarily focuses on Lambda-DynamoDB integration and CRUD execution logic.

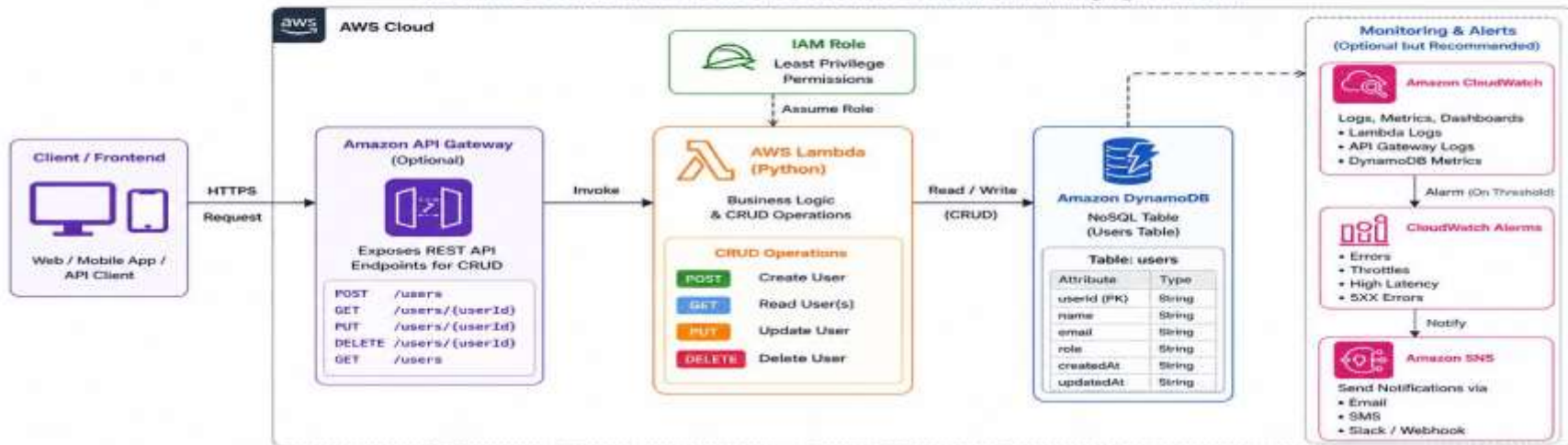
- 1 Client/Frontend sends HTTPS requests to API Gateway endpoints
- 2 API Gateway routes requests to Lambda function based on HTTP method
- 3 Lambda function processes business logic and CRUD operations
- 4 Lambda interacts with DynamoDB for data persistence using Boto3
- 5 CloudWatch monitors Lambda execution and logs performance metrics
- 6 SNS sends notifications for errors, throttles, and critical events

Architecture Diagram

Production-Style Serverless Backend using AWS Lambda & DynamoDB

Full CRUD Operations (Create, Read, Update, Delete)

A scalable, secure and cost-effective serverless backend for managing user data.



Architecture Flow

- 1 Client sends HTTPS request to API Gateway (optional)
- 2 API Gateway forwards request to AWS Lambda
- 3 Lambda processes request based on HTTP method
- 4 Lambda performs CRUD operation on DynamoDB
- 5 Response is returned to client via API Gateway

Key Features

- ✓ Full CRUD operations
- ✓ Serverless & fully managed
- ✓ Auto scaling & high availability
- ✓ Pay-per-request (cost effective)
- ✓ Secure with IAM least privilege
- ✓ No server management

Technologies Used

- ✓ AWS Lambda (Python 3.x)
- ✓ Amazon API Gateway (REST API)
- ✓ Amazon DynamoDB (NoSQL)
- ✓ Amazon CloudWatch (Monitoring)
- ✓ Amazon SNS (Notifications)
- ✓ IAM (Roles & Policies)

Notes

- API Gateway is optional. You can test Lambda directly using the Lambda console.
- CloudWatch & SNS add observability and alerting for production readiness.
- This architecture supports full CRUD operations with a clean, scalable, and serverless design.

Current Implementation: Lambda + DynamoDB integration with CRUD operations (tested via Lambda console).

Possible Enhancements: Add API Gateway, Enable CloudWatch Alarms, Add SNS notifications, Add Cognito for authentication.

API Gateway Workflow

API Gateway acts as the entry point for all client requests, exposing RESTful endpoints:

HTTP Method	Endpoint	Action
POST	/users	Create User
GET	/users/{userId}	Read User
PUT	/users/{userId}	Update User
DELETE	/users/{userId}	Delete User
GET	/users	List All Users

Lambda CRUD Workflow

API Gateway routes all HTTP requests to a single Lambda function, which internally separates CRUD operations using HTTP methods.

```
def lambda_handler(event, context):
    http_method = event['httpMethod']

    if http_method == 'POST':
        # Create user logic
    elif http_method == 'GET':
        # Read user logic
    elif http_method == 'PUT':
        # Update user logic
    elif http_method == 'DELETE':
        # Delete user logic
```

Key Concept

Single Function, Multiple Operations

Instead of creating separate Lambda functions for each CRUD operation, we use a single function that routes internally based on the HTTP method.

This approach:

- Reduces complexity
- Simplifies deployment
- Centralizes business logic
- Maintains clean separation through code structure

DynamoDB Integration



DynamoDB serves as the NoSQL database, providing fast and scalable data storage:

Table Structure: users-table

Attribute	Type	Description
userId (PK)	String	Unique identifier
name	String	User's full name
role	String	User role/type

Key Features

- Serverless & Auto-scaling
Automatically scales capacity based on traffic
- Single-Digit Millisecond Latency
Consistent fast performance at any scale
- Flexible Schema
NoSQL allows easy schema evolution
- Built-in Security
IAM-based access control and encryption

CRUD Operations Explained

CREATE (POST)

Adds a new user record to DynamoDB

```
table.put_item(Item={...})
```

READ (GET)

Retrieves user data by userId

```
table.get_item(Key={'userId': id})
```

UPDATE (PUT)

Modifies existing user attributes

```
table.update_item(Key=...,  
UpdateExpression=...)
```

DELETE (DELETE)

Removes user record from database

```
table.delete_item(Key={'userId': id})
```

IAM Security & Permissions



AWS IAM (Identity and Access Management) controls which services can communicate:

Security Principle: Least Privilege Access



Lambda Execution Role

- Read/Write to DynamoDB users-table
- Write logs to CloudWatch
- Access to SNS for notifications



API Gateway Permissions

- Invoke Lambda function
- Pass request context to Lambda



DynamoDB Access Control

- Only Lambda can access table
- Encryption at rest enabled

Monitoring: CloudWatch & SNS



Amazon CloudWatch

Metrics Tracked:

- Lambda invocations count
- Execution duration
- Error rates and types
- Throttling events
- DynamoDB read/write units

Logs:

- Complete execution traces
- Request and response data
- Error stack traces



Amazon SNS

Notification Flow:

CloudWatch Alarm



SNS Topic Trigger



Notifications via:

- Email
- SMS
- Slack/Webhook

SNS workflow included as a production-style monitoring concept extension.

Step-by-Step Implementation

- | | | |
|---|-------------------------------------|--|
| 1 | Create DynamoDB Table | Set up users-table with userId as partition key |
| 2 | Configure IAM Role | Create execution role with DynamoDB and CloudWatch permissions |
| 3 | Write Lambda Function | Develop Python handler with CRUD logic using Boto3 |
| 4 | Deploy Lambda Function | Upload code and attach IAM role to function |
| 5 | Test Lambda Directly | Verify function execution through Lambda console |
| 6 | Validate in DynamoDB | Confirm data persistence and operations |
| 7 | Setup API Gateway (Optional) | Create REST API endpoints for production use |
| 8 | Configure Monitoring | Enable CloudWatch logs and SNS notifications |

Implementation: DynamoDB Table Setup

✓ DynamoDB table 'users-table' successfully created

The screenshot displays the AWS Management Console interface for a DynamoDB table named 'users-table'. The browser address bar shows the URL: `ap-south-1.console.aws.amazon.com/dynamodbv2/home?region=ap-south-1#table?name=users-table`. The console header includes the AWS logo, a search bar, and the user's name 'Adithyan Sivaraman T (4173-3166-7394)'. The left-hand navigation pane lists various services, with 'DynamoDB' selected. Under 'DynamoDB', the 'Tables' section is active, showing a list of tables with 'users-table' highlighted. The main content area is titled 'users-table' and includes a timestamp 'February 23, 2026, 23:22 (UTC+5:30)'. It features several tabs: 'Settings' (selected), 'Indexes', 'Monitor', 'Global tables', 'Backups', 'Exports and streams', and 'Permissions'. A prominent blue alert box states: 'Protect your DynamoDB table from accidental writes and deletes. When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)'. Below this, the 'General information' section provides details: Partition key is 'userid (String)', Sort key is '-', Capacity mode is 'On-demand', Item count is '0', Table status is 'Active', and Table size is '0 bytes'. Other fields include 'Alarms' (No active alarms), 'Average item size' (0 bytes), 'Point-in-time recovery (PITR)' (OFF), 'Resource-based policy' (Not active), and 'Amazon Resource Name (ARN)' (`arn:aws:dynamodb:ap-south-1:417311687394:table/users-table`). The bottom of the console shows a footer with 'CloudShell', 'Feedback', 'Console Mobile App', copyright information, and links for 'Privacy', 'Terms', and 'Cookie preferences'. The system clock at the bottom right indicates '23:23 23-02-2026'.

Implementation: Lambda Function Config

✓ Lambda function 'dynamodb-crud-fn' configured with IAM permissions

The screenshot displays the AWS Lambda console interface for the function 'dynamodb-crud-fn'. The browser address bar shows the URL: `ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/dynamodb-crud-fn/subtab=permissions/sub-code`. The console header includes the AWS logo, a search bar, and navigation links for IAM, Aurora and RDS, and DynamoDB. The breadcrumb trail indicates the path: `Lambda > Functions > dynamodb-crud-fn`.

The function overview section shows the function name 'dynamodb-crud-fn' with a diagram icon and a 'Layers' section indicating '(0)' layers. There are buttons for 'Add trigger' and 'Add destination'. On the right, there are buttons for 'Throttle', 'Copy ARN', and 'Actions'. Below these are buttons for 'Export to infrastructure Composer' and 'Download'.

The 'Description' section on the right provides details about the function:

- Description:** -
- Last modified:** 10 minutes ago
- Function ARN:** `arn:aws:lambda:ap-south-1:417311687394:function:dynamodb-crud-fn`
- Function URL:** [Info](#)

The bottom navigation bar includes tabs for 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Code source' section at the bottom shows buttons for 'Open in Visual Studio Code' and 'Upload from'.

The footer of the console displays the copyright notice: '© 2026, Amazon Web Services, Inc. or its affiliates.' along with links for 'Privacy', 'Terms', and 'Cookie preferences'. The system clock at the bottom right shows the date '23-02-2026' and time '23:23'.

Implementation: Lambda Testing

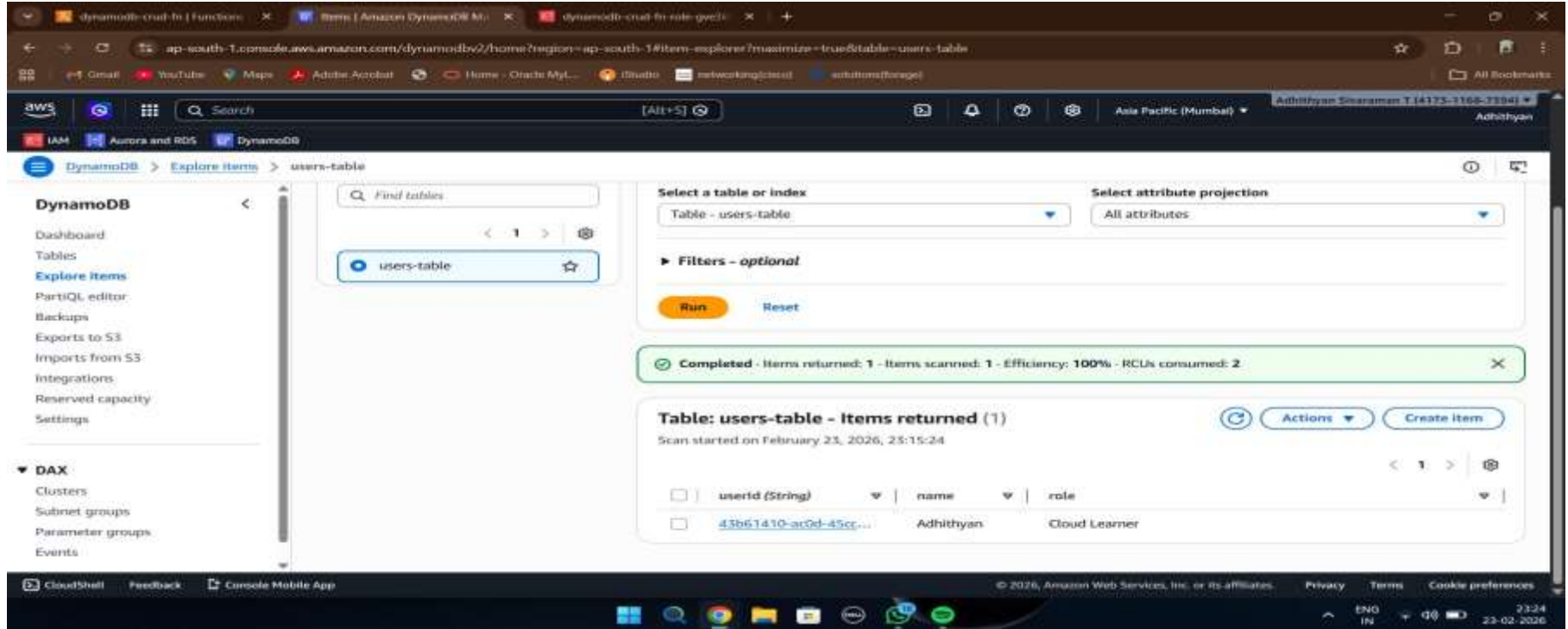
✓ Lambda function successfully executed with status code 200

The screenshot displays the AWS Lambda console interface. The top navigation bar shows the user is logged in as 'Adhithyan Sivaraman T (4175-3166-7194)' in the 'Asia Pacific (Mumbai)' region. The main content area is titled 'Lambda > Functions > dynamodb-crud-fn'. On the left sidebar, the 'FUNCTIONS' section is expanded, showing the 'dynamodb-crud-fn' function. The 'DEPLOY' section shows the function is 'Current' and has a 'Deploy' button. The 'TEST EVENTS' section shows a test event named 'Inserting-values' with a status of 'Succeeded'. The 'ENVIRONMENT VARIABLES' section is also visible. The main pane shows the code for the 'lambda_function.py' file, which imports 'json', 'boto3', and 'uuid', and defines a 'lambda_handler' function that inserts a user into a DynamoDB table. The 'Execution Results' pane at the bottom shows the status 'Succeeded', the test event name 'Inserting-values', and the response:

```
{  "statusCode": 200,  "body": "\\nUser inserted successfully\\n"}
```

Implementation: Data Validation

✓ User record successfully inserted and visible in DynamoDB table



The screenshot displays the AWS Management Console interface for the 'users-table' in the 'ap-south-1' region. The left sidebar shows the 'DynamoDB' navigation menu with options like Dashboard, Tables, Explore Items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The 'Explore Items' section is active, showing a search bar and a list of tables, with 'users-table' selected. The main content area shows the 'Select a table or index' dropdown set to 'Table - users-table' and the 'Select attribute projection' dropdown set to 'All attributes'. Below these, there are 'Filters - optional' and 'Run' and 'Reset' buttons. A green status bar indicates 'Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2'. The table 'users-table' shows one item returned, with columns 'userId (String)', 'name', and 'role'. The item has a 'userId' of '45b61410-arc0d-45cc...' and a 'name' of 'Adhithyan'.

DynamoDB

Dashboard
Tables
Explore Items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings

DAX
Clusters
Subnet groups
Parameter groups
Events

Find tables

users-table

Select a table or index
Table - users-table

Select attribute projection
All attributes

Filters - optional

Run Reset

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2

Table: users-table - Items returned (1)
Scan started on February 23, 2026, 23:15:24

	userId (String)	name	role
	45b61410-arc0d-45cc...	Adhithyan	Cloud Learner

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

23:24
23-02-2026

Complete Request Flow

- ➔ 1 Client sends HTTP request (POST /users)
- ➔ 2 API Gateway receives and validates request
- ➔ 3 API Gateway invokes Lambda function
- ➔ 4 Lambda checks event['httpMethod'] → 'POST'
- ➔ 5 Lambda executes create user logic
- ➔ 6 Boto3 writes data to DynamoDB table
- ➔ 7 DynamoDB returns success confirmation
- ➔ 8 Lambda returns response to API Gateway
- ➔ 9 API Gateway returns HTTP 200 to client
- ➔ 10 CloudWatch logs execution metrics

Key Concepts Learned



Serverless Computing

- Event-driven architecture
- No server provisioning
- Automatic scaling



NoSQL Database Design

- Partition key selection
- Flexible schema structure
- DynamoDB best practices



API Design Patterns

- RESTful endpoint mapping
- HTTP method routing
- Request/response handling



AWS Security

- IAM role-based access
- Least privilege principle
- Service-to-service auth



Frontend Integration Flow

- S3 + CloudFront hosting
- API Gateway connection
- Serverless full-stack pattern



Python & Boto3

- AWS SDK usage
- DynamoDB operations
- Error handling

Challenges & Solutions

IAM Permission Configuration

Carefully studied AWS IAM documentation and applied least privilege principle to grant only necessary permissions

Lambda-DynamoDB Integration

Used Boto3 SDK to establish connection and learned proper error handling for database operations

HTTP Method Routing Logic

Implemented clean if-elif structure to separate CRUD operations within single Lambda function

Testing Serverless Functions

Leveraged Lambda console test events to validate function behavior before integration

Future Enhancements

Potential improvements to make this architecture production-ready:



API Gateway Integration

Deploy REST API endpoints for external client access



Input Validation

Add comprehensive request validation and sanitization



Rate Limiting

Configure API Gateway throttling and quotas



DynamoDB Backup

Enable point-in-time recovery and automated backups



Frontend Integration

Connect S3-hosted frontend via CloudFront



Authentication & Authorization

Implement AWS Cognito for user authentication



Error Handling

Implement robust error responses and logging



CloudWatch Alarms

Set up proactive monitoring and alerting



CI/CD Pipeline

Automate deployment using AWS CodePipeline



Multi-Environment Setup

Create separate dev, staging, and production environments

Final Outcome



Successfully Implemented

A working serverless backend demonstrating production-style architecture patterns with Lambda and DynamoDB integration. The implementation validates core concepts through hands-on practice.

What Was Achieved:

- ✓ DynamoDB table successfully created and configured
- ✓ Lambda function deployed with proper IAM permissions
- ✓ CRUD operations validated through Lambda console testing
- ✓ Data persistence confirmed in DynamoDB
- ✓ Lambda execution logs observed using CloudWatch
- ✓ Understanding of serverless architecture patterns gained

Key Takeaways



Serverless Simplifies Infrastructure

No server management means focusing on code and business logic instead of infrastructure operations



Auto-Scaling is Powerful

Applications automatically handle traffic spikes without manual intervention or pre-provisioning



NoSQL Flexibility

DynamoDB's flexible schema allows rapid iteration and easy data model evolution



Security Through IAM

Proper IAM configuration is crucial for secure service-to-service communication



Learning by Building



Conclusion

This project demonstrates a practical understanding of production-style serverless architecture through hands-on implementation. While the architecture diagram represents a complete production system, the current implementation focuses on core concepts:

- Lambda + DynamoDB integration
- CRUD operation implementation
- IAM security configuration
- CloudWatch monitoring

This project helped bridge the gap between traditional backend architecture and modern serverless cloud-native application design.

This learning journey continues with deeper explorations into AWS services, DevOps practices, and cloud-native application development.